Proxy Facade Builder: Enriching Facade Construction for Pointer-Semantics Polymorphism

Document #:	P3584R1 [Latest] [Status]
Date:	2025-10-06
Project:	Programming Language C++
Audience:	LEWGI LEWG

Reply-to:	Mingxin Wang <mingxwa@outlook.com>

Contents

Abstract

This paper proposes `basic_facade_builder`, a fluent, zero-runtime-overhead facility to assemble *facades* (the compile-time descriptions of operation sets, reflection hooks, and storage / lifetime constraints) used by the pointer-semantics polymorphism facility. The builder standardizes reusable composition patterns (including *skills*), enforces idempotent merging of conventions and reflections, and exposes explicit non-type template parameters for size, alignment, and constraint levels. It does not alter the underlying `proxy` semantics; intermediate builder types are purely compile-time and can be optimized away.

§ 1 History

§ 1.1 Changes from P3584R0

— Replaced the single `proxiable_ptr_constraints` aggregate template parameter with explicit non-type template parameters (`MaxSize`, `MaxAlign`, `Copyability`, `Relocatability`, `Destructibility`) on `basic_facade_builder`, aligning with the core proposal's removal of that aggregate facility.

— Updated defaults: relocatability now defaults to `trivial` (rather than the earlier wording that implied `nothrow`) to mirror the current implementation practice and optimize move pathways; clarified all fallback defaults in one table.

— Revised semantics of `add_facade`: it now (optionally) injects a *direct* convention for substitution via `substitution_dispatch` (opt-in boolean). The prior upward/implicit conversion wording is superseded.

— Added the new API `basic_facade_builder::add_skill` (skill composition mechanism) and described its role in reducing boilerplate and encouraging facade modularity.

— Clarified idempotence rules: merging duplicate conventions / reflections / skills is well-defined and does not affect ABI or codegen.

— Added rationale for explicit opt-in to substitution to balance backward compatibility with minimal binary size.

§ 2 Introduction

This paper extends the pointer-semantics runtime polymorphism facility (P3086) by standardizing a *facade construction DSL* (`basic_facade_builder`). The goal is to make authoring facades (compile-time descriptions of operation sets, reflection hooks, and storage / lifetime constraints) concise, declarative, and mechanically verifiable while preserving the minimal core surface standardized for the primary `proxy` facility.

A facade determines which *conventions* (dispatch families) and *reflections* (lightweight type metadata) a `proxy<F>` exposes, and encodes constraint levels (copy, relocation, destruction) plus storage limits. `basic_facade_builder` offers a chained, zero-runtime-overhead API to assemble these characteristics from reusable *skills* and previously defined facades.

§ 3 Motivation and scope

§ 3.1 Problem

Authoring a facade manually requires spelling out tuple aggregations and constraint constants. This is error prone and obscures intent, especially when composing capabilities from multiple sources (e.g. formatting, RTTI, view / weak adaptation, callable overload sets, substitution pathways for versioning).

§ 3.2 Goals

1. Expressively assemble facade operation sets from reusable building blocks (skills, sub-facades) with linear, readable syntax.
2. Make capability composition *idempotent*: duplicate additions are harmless.
3. Allow opt-in backward compatible substitution (capability downgrade) without forcing all facades to pay for that path.
4. Encode storage / lifetime constraints early so downstream `proxy` instantiations optimize layout (SBO vs pointer) predictably.
5. Provide a portable pattern that compilers can optimize aggressively (no runtime branching, constant expression propagation).

§ 3.3 Non-Goals

— Automatic facade inference from arbitrary sets of functions (future reflection-powered direction).

— Standardization of auxiliary skill catalogs beyond those needed to demonstrate composition.

— Mandating specific code generation heuristics (embedding thresholds remain implementation strategy).

§ 4 Considerations and design decisions

This section summarizes the principal design choices that motivated the shape of the facility.

§ 4.1 Fluent facade construction DSL

`basic_facade_builder` exposes a fluent chain of alias templates that transform a phantom parameter pack (conventions, reflections, constraint values) into a concrete facade type at the final `::build`. Each intermediate type is distinct and discarded at runtime; only the resulting facade participates in object code generation. Users never spell the underlying template parameters directly; they always start from `facade_builder` and compose.

§ 4.2 Explicit constraint parameters

The removal of the single aggregate constraints struct improves clarity, constant propagation, and reduces accidental partial initialization. Each dimension (size, alignment, copyability, relocatability, destructibility) composes via `min` or `max` monotonic rules ensuring associativity and order independence.

§ 4.3 Idempotent merging

Repeated additions of the same (dispatch type, directness) pair or reflection type unify their overload sets and drop duplicates. This property enables layering skills and facades without defensive checks or order sensitivity.

§ 4.4 Substitution as an opt-in convention

Backward compatibility and capability downgrades are implemented through `substitution_dispatch`. Rather than always embedding a conversion path (increasing metadata and potentially code size), `add_facade<F, true>` inserts a direct substitution convention with overloads returning `proxy<G>` for eligible downgrades. Developers choose where API versioning or legacy interop justifies the cost.

§ 4.5 Skills for higher-level composition

`add_skill<Skill>` injects a reusable *micro-builder*: a template taking the current builder and returning an augmented builder. Skills may internally chain other builder aliases (e.g. restrict layout, add reflection & formatting conventions, add view/weak adaptation support). This keeps user facades terse while centralizing best practice combinations.

§ 4.6 Default semantics and rationale

The default constraint constants are intentionally chosen so that a facade created directly from `facade_builder::build` works for the predominant set of pointer-like ownership handles (unique pointer, many shared pointer implementations, offset / arena pointers) **while remaining small enough to encourage stack placement and enable small buffer optimization (two pointer-sized words)**. Relocatability defaults to `trivial` to unlock the shortest move paths and permit aggressive optimizer reasoning (bitwise relocation) while still allowing users to raise copyability explicitly when deep copies are semantically desired. Destructibility defaults to `nothrow` to favor predictable unwinding and simplify generated exception paths. Copyability is opt-in (`none` by default) to avoid silent O(N) copies of expensive handles. These defaults have shipped in production deployments and represent an empirically balanced baseline rather than arbitrary sentinel resolution.

Dimension	Resolved Default (`build`)
`max_size`	`sizeof(void*) * 2`
`max_align`	`alignof(void*)`
`copyability`	`constraint_level::none`
`relocatability`	`constraint_level::trivial`
`destructibility`	`constraint_level::nothrow`

§ 4.7 Substitution rationale

Embedding substitution only when requested preserves the *pay for what you use* principle: many facades are leaf abstractions or internal to a single module boundary and never require downgrade conversion. Making substitution explicit surfaces API boundary decisions and keeps tiny facades at minimal footprint.

§ 4.8 Skill pattern rationale

Skills encapsulate proven combinations (e.g. format + RTTI + slim shared semantics) in a compile-time pipeline. Because each step is a pure type transformation, compilers discard unused intermediate instantiations, providing the readability of a fluent DSL with the optimization of hand-written manual facades.

§ 4.9 Error handling and diagnostics

Constraint elevation and convention merging are purely type-level; failures manifest as concept substitution errors naming the missing requirement (e.g. violation of an overload's required signature or constraint level). This matches the diagnostic style established in the core `proxy` facility.

§ 4.10 Performance considerations

The builder itself imposes no runtime overhead; all decisions (embedding vs table, SBO viability, trivial relocation) depend only on the final facade constants and convention set

size. Explicit early restriction (`restrict_layout`) enables smaller `proxy` objects and tighter cache footprints. Opting into substitution adds only the dispatch entries needed for conversion; unused pathways are eliminated by the linker and dead code elimination.

§ 5 Technical specifications

This section contains wording intended for integration with the primary `proxy` wording. Add the following to <memory>.

§ 5.1 Feature test macro

In *[version.syn]*, update:

```
#define __cpp_lib_proxy YYYYMML // also in <memory>
```

The value is updated to the date of adoption of this paper.

§ 5.2 Header <memory> synopsis (additions)

```
namespace std {

  // exposition-only sentinels
  constexpr size_t default-size = numeric_limits<size_t>::max();  // expo-
        sition only
  constexpr constraint_level default-cl = static_cast<constraint_level>(
        numeric_limits< underlying_type_t<constraint_level> >::min()); //
        exposition only

  template<class Cs, class Rs,
           size_t MaxSize, size_t MaxAlign,
           constraint_level Copyability,
           constraint_level Relocatability,
           constraint_level Destructibility>
    class basic_facade_builder; // no value construction

  using facade_builder = basic_facade_builder< tuple<>, tuple<>,
      default-size, default-size,
      default-cl, default-cl, default-cl >;

} // namespace std
```

§ 5.3 Class template `basic_facade_builder`

For a specialization `basic_facade_builder<Cs, Rs, MaxSize, MaxAlign, Copyability, Relocatability, Destructibility>`:

§ 5.3.1 Member alias templates

Each alias template denotes another specialization of `basic_facade_builder` whose template arguments are derived from those of the current specialization as specified below. Let

`F` denote any resulting facade type produced by a `build` member from some builder specialization.

```
template<class D, class... Os> requires (sizeof...(Os) > 0)
using add_convention = see below;          // indirect by default
template<class D, class... Os> requires (sizeof...(Os) > 0)
using add_indirect_convention = see below;
template<class D, class... Os> requires (sizeof...(Os) > 0)
using add_direct_convention = see below;

template<class R>
using add_reflection = see below;          // indirect by default
template<class R>
using add_indirect_reflection = see below;
template<class R>
using add_direct_reflection = see below;

template<facade G, bool WithSubstitution = false>
using add_facade = see below;

template< template<class> class Skill > requires(see below)
using add_skill = Skill< basic_facade_builder >;

template<size_t PtrSize, size_t PtrAlign = see below>
  requires (has_single_bit(PtrAlign) && PtrSize % PtrAlign == 0)
using restrict_layout = see below;

template<constraint_level CL>
using support_copy = see below;
template<constraint_level CL>
using support_relocation = see below;
template<constraint_level CL>
using support_destruction = see below;

using build = see below;
```

§ 5.3.2 Effects

Unless otherwise specified, alias template instantiations do not modify any template argument other than those stated.

`add_convention` / `add_indirect_convention` / `add_direct_convention`:

— Form an exposition-only convention type `IC` where `IC::dispatch_type` is `D`, `IC::is_direct` is `false` (indirect variants) or `true` (direct variants), and `IC::overload_types` is a tuple-like type consisting of the distinct types in `Os` after facade-aware substitution of overload signatures. If `Cs` already contains a convention `IC2` with the same `is_direct` and `dispatch_type`, the overload set is replaced by the union (duplicates removed) and the number of elements of `Cs` does not change; otherwise `IC` is appended. `add_convention` is equivalent to `add_indirect_convention`.

`add_reflection` / `add_indirect_reflection` / `add_direct_reflection`:

— Form an exposition-only reflection descriptor `Refl` where `Refl::reflector_type` is `R` and `Refl::is_direct` corresponds to the directness variant. If a descriptor equivalent to `Refl` already appears in `Rs` it is not duplicated. `add_reflection` is equivalent to `add_indirect_reflection`.

`add_facade<G, WithSubstitution>`:

— Merges `typename G::convention_types` and `typename G::reflection_types` as if by repeated application of the corresponding add operations (preserving idempotence rules). Sets `MaxSize = min(MaxSize, G::max_size)` and `MaxAlign = min(MaxAlign, G::max_align)`. Sets each of `Copyability`, `Relocatability`, `Destructibility` to `max(Current, G::corresponding)` respectively. If `WithSubstitution` is `true`, additionally merges a direct convention whose dispatch type is `substitution_dispatch` with an exposition-only overload set permitting substitution to eligible facades reachable from `G` (including downgrades) and whose overloads uniformly propagate null state.

`add_skill<Skill>`:

— Equivalent to `Skill<basic_facade_builder>` where `Skill<basic_facade_builder>` must be a specialization of `basic_facade_builder`.

`restrict_layout<PtrSize, PtrAlign>`:

— Sets `MaxSize = min(MaxSize, PtrSize)` and `MaxAlign = min(MaxAlign, PtrAlign)`. The default for `PtrAlign` is the maximum alignment not exceeding `alignof(max_align_t)` that is valid for objects of size `PtrSize`.

`support_copy<CL>` / `support_relocation<CL>` / `support_destruction<CL>`:

— Replace the corresponding template argument with `max(Current, CL)`.

`build`:

— Names a facade type `F` with member types `convention_types = Cs`, `reflection_types = Rs` and static data members:

- `max_size = (MaxSize == default-size ? sizeof(void*) * 2u : MaxSize)`
- `max_align = (MaxAlign == default-size ? alignof(void*) : MaxAlign)`
- `copyability = (Copyability == default-cl ? constraint_level::none : Copyability)`
- `relocatability = (Relocatability == default-cl ? constraint_level::trivial : Relocatability)`
- `destructibility = (Destructibility == default-cl ? constraint_level::nothrow : Destructibility)`

§ 5.3.3 Remarks

Adding duplicate conventions, reflections, or skills is well-defined and has no observable effect other than unifying overload sets. The primary template of `basic_facade_builder` has no constructors (value construction is ill-formed).

§ 6 Summary

`basic_facade_builder` provides a principled, declarative layer for constructing facades: expressive, idempotent, opt-in for backward compatibility, and fully aligned with pointer-semantics performance goals. It lowers adoption friction for the core `proxy` model while preserving its minimal runtime cost characteristics.